

Национальный исследовательский ядерный университет «МИФИ»

Институт интеллектуальных кибернетических систем

Кафедра №12 «Компьютерные системы и технологии»



ОТЧЕТ

О выполнении лабораторной работы № 5 «Работа с массивами структур. Исследование методов сортировки массивов»

Студент: Сейтназаров Д.Н.

Группа: Б24-703

Преподаватель: Храпов А.С.

Москва – 2024

1. Формулировка индивидуального задания

Вариант №195.

Структура данных

Запись в журнале событий:

- идентификатор (целое число)
- уровень важности (целое число, обозначающее один из следующих уровней: debug, info, warn, error, fatal)
- текст (строка произвольной длины).

Алгоритмы сортировки

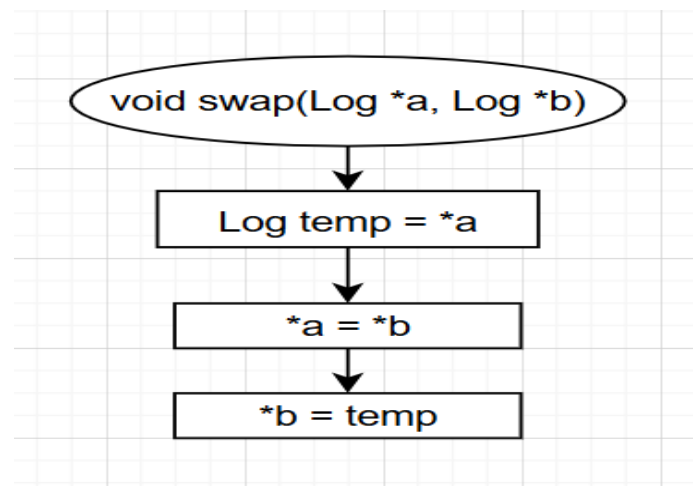
1. Шейкерная сортировка (Shaker sort).
2. Двухсторонняя сортировка выбором (Double selection sort).

2. Описание использованных типов данных

При выполнении данной лабораторной работы использовался встроенный тип данных `int`, предназначенный для работы с целыми числами.

3. Описание использованного алгоритма

Рис. 1: Блок-схема алгоритма работы функции `swap ( )`



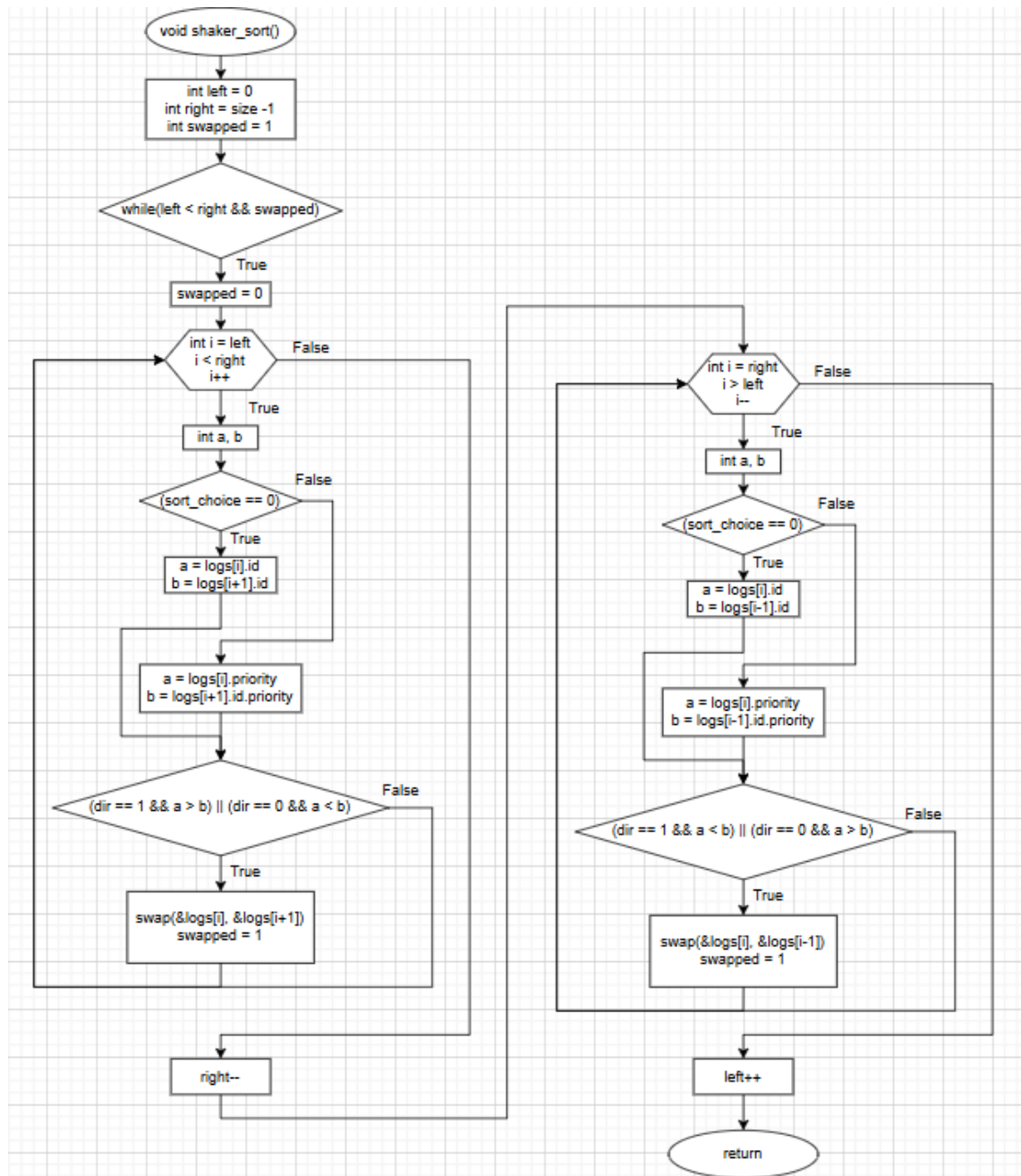


Рис. 2: Блок-схема алгоритма работы функции `Shaker_sort()`



4. Исходные коды разработанных программ

Листинг 1: Исходные коды программы main1 (файл: main.c)

```
1 #include <stdlib.h>
2 #include <string.h>
3 #include <unistd.h>
4 #include "heads.h"
5
6
7 int main(int argc, char *argv[]) {
8     Log logs[MAXREC]; // массив структур Log для хранения записей логов
9     int log_count = 0; // хранение количества загруженных логов
10
11     char *input_file = NULL;
12     char *output_file = NULL;
13     int binary_input = 0, binary_output = 0;
14     int opt;
15     // все опции обработаны - getopt вернет -1
16     while ((opt = getopt(argc, argv, "i:o:bt")) != -1) {
17         switch (opt) {
18             case 'i': input_file = optarg; break;
19             case 'o': output_file = optarg; break;
20             case 'b': binary_input = 1; break;
21             case 't': binary_output = 1; break;
22             default:
23                 fprintf(stderr, "Help: %s [-i файл_входа] [-o файл_выхода] "
24                     " [-b (бинарный вход)] [-t (бинарный выход)]\n", argv[0]);
25                 exit(EXIT_FAILURE);
26         }
27     }
28     if (input_file) {
29         if (binary_input) {
30             log_count = from_binary(input_file, logs, MAXREC);
31         } else {
32             log_count = from_text(input_file, logs, MAXREC);
33         }
34     } else {
35         log_count = from_stdin(logs, MAXREC);
36     }
37
38     if (output_file) {
39         if (binary_output) {
40             to_binary(output_file, logs, log_count);
41         } else {
42             to_text(output_file, logs, log_count);
43         }
44     } else {
45         print_logs(logs, log_count);
46     }
47
48     return 0;
49 }
```

Листинг 2: Исходные коды программы main2 (файл: main.c)

```
1 #include <stdlib.h>
2 #include <unistd.h>
3 #include "heads.h"
4 int main(int argc, char *argv[]) {
5     Log logs[MAXREC]; // массив структур Log для хранения записей логов
6     int log_count = 0; // хранение количества загруженных логов
7     char *input_file = NULL;
8     char *output_file = NULL;
9     int binary_input = 0, binary_output = 0;
10    int sort_method = 0, sort_choice = 0, dir = 1; // direction - направление
11    int opt;
12    while ((opt = getopt(argc, argv, "i:o:btm:f:d")) != -1) {
13        switch (opt) {
14            case 'i': input_file = optarg; break;
15            case 'o': output_file = optarg; break;
16            case 'b': binary_input = 1; break;
17            case 't': binary_output = 1; break;
18            case 'm': sort_method = atoi(optarg); break;
19            case 'f': sort_choice = atoi(optarg); break;
20            case 'd': dir = 0; break;
21            default:
22                fprintf(stderr, "Help: %s [-i файл_входа] [-o файл_выхода] "
23                    "[-b (бин. вход)] [-t (бин. выход)] [-f (0-id, 1-priority)] "
24                    "[-d (0-убыв, 1-возр)]\n", argv[0]);
25                exit(EXIT_FAILURE);
26        }
27    }
28    if (input_file) {
29        if (binary_input) {
30            log_count = from_binary(input_file, logs, MAXREC);
31        } else {
32            log_count = from_text(input_file, logs, MAXREC);
33        }
34    } else {
35        log_count = from_stdin(logs, MAXREC);
36    }
37    if (sort_method == 0) {
38        shaker_sort(logs, log_count, sort_choice, dir);
39    } else if (sort_method == 1) {
40        double_selection_sort(logs, log_count, sort_choice, dir);
41    }
42    if (output_file) {
43        if (binary_output) {
44            to_binary(output_file, logs, log_count);
45        }
46        else {
47            to_text(output_file, logs, log_count);
48        }
49    } else {
50        print_logs(logs, log_count);
51    }
52    return 0;
53 }
```

Листинг 3: Исходные коды программы main3 (файл: main.c)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <time.h>
6 #include "heads.h"
7
8 int main(int argc, char *argv[]) {
9     srand(time(NULL));
10
11     Log *logs = malloc(MAXREC * sizeof(Log));
12     if (!logs) {
13         perror("Failed to allocate!!!!");
14         return EXIT_FAILURE;
15     }
16     int log_count = 0;
17     char *input_file = NULL;
18     char *output_file = NULL;
19     int binary_input = 0, binary_output = 0;
20     int sort_choice = 0, dir = 1;
21     int opt;
22     while ((opt = getopt(argc, argv, "i:o:btfd")) != -1) {
23         switch (opt) {
24             case 'i': input_file = optarg; break;
25             case 'o': output_file = optarg; break;
26             case 'b': binary_input = 1; break;
27             case 't': binary_output = 1; break;
28             case 'f': sort_choice = atoi(optarg); break;
29             case 'd': dir = 0; break;
30             default:
31                 fprintf(stderr, "Help: %s [-i файл_входа] [-o файл_выхода] [-b
32 (бин. вход)] [-t (бин. выход)] "
33                     " [-f (0-id, 1-priority)] [-d (0-убыв, 1-возр)]\n",
34 argv[0]);
35                 free(logs);
36                 exit(EXIT_FAILURE);
37         }
38     }
39     if (input_file) {
40         if (binary_input) {
41             log_count = from_binary(input_file, logs, MAXREC);
42         }
43         else {
44             log_count = from_text(input_file, logs, MAXREC);
45         }
46     }
47     else {
48         log_count = from_stdin(logs, MAXREC);
49     }
50
51     printf("\n");
52     sort_time(qsort_sort, logs, log_count, sort_choice, dir, "qsort");
53     sort_time(shaker_sort, logs, log_count, sort_choice, dir, "shaker sort");
54     sort_time(double_selection_sort, logs, log_count, sort_choice, dir, "double
55 selection sort");
56
57     if (output_file) {
```

```

58     if (binary_output) {
59         to_binary(output_file, logs, log_count);
60     }
61     else {
62         to_text(output_file, logs, log_count);
63     }
64 }
65 else {
66     print_logs(logs, log_count);
67 }
free(logs);
return 0;
}

```

Листинг 4: Исходные коды программы rw (файл: rw.c)

```

1 #include <stdlib.h>
2 #include <string.h>
3 #include "heads.h"
4
5 const char* priority_to_string(int priority) {
6     switch (priority) {
7         case 0: return "debug";
8         case 1: return "info";
9         case 2: return "warn";
10        case 3: return "error";
11        case 4: return "fatal";
12        default: return "invalid";
13    }
14 }
15
16 int from_stdin(Log *logs, int max_size) { // MAXREC
17     int count; // сколько записей хочешь ?
18     printf("Enter count of logs: ");
19     if (scanf("%d", &count) != 1 || count <= 0 || count > max_size) {
20         fprintf(stderr, "Error: invalid count of logs!\n");
21         return 0;
22     }
23     getchar(); // очистка буфера
24     for (int i = 0; i < count; i++) {
25         printf("Enter id, priority (0-4) и message: ");
26         if (scanf("%d %d", &logs[i].id, &logs[i].priority) != 2) {
27             fprintf(stderr, "Error: invalid input!\n");
28             return i;
29         }
30         getchar();
31         fgets(logs[i].message, MAXCHAR, stdin);
32         logs[i].message[strcspn(logs[i].message, "\n")] = '\0';
33     }
34     return count;
35 }
36
37 int from_text(const char *filename, Log *logs, int max_size) {
38     FILE *file = fopen(filename, "r");
39     if (!file) {
40         perror("Error: can't open file");
41         return 0;
42     }

```



```

43     int count = 0;
44     while (count < max_size && fscanf(file, "%d %d", &logs[count].id,
45 &logs[count].priority) == 2) {
46         fgetc(file);
47         fgets(logs[count].message, MAXCHAR, file);
48         logs[count].message[strcspn(logs[count].message, "\n")] = '\0';
49         count++;
50     }
51     fclose(file);
52     return count;
53 }
54
55 int from_binary(const char *filename, Log *logs, int max_size) {
56     FILE *file = fopen(filename, "rb");
57     if (!file) {
58         perror("Error: can't open file");
59         return 0;
60     }
61     int count = fread(logs, sizeof(Log), max_size, file);
62     fclose(file);
63     return count;
64 }
65
66 void print_logs(const Log *logs, int count) {
67     printf("\nLogs:\n");
68     for (int i = 0; i < count; i++) {
69         printf("%d [%s] %s\n", logs[i].id, priority_to_string(logs[i].priority),
70 logs[i].message);
71     }
72 }
73
74 void to_text(const char *filename, const Log *logs, int count) {
75     FILE *file = fopen(filename, "w");
76     if (!file) {
77         perror("Error: can't open file");
78         return;
79     }
80     char buffer[512];
81
82     for (int i = 0; i < count; i++) {
83         snprintf(buffer, sizeof(buffer), "%d %d %s\n", logs[i].id,
84 logs[i].priority, logs[i].message); // буфер в файл
85         fputs(buffer, file);
86     }
87
88     fclose(file);
89 }
90
91 void to_binary(const char *filename, const Log *logs, int count) {
92     FILE *file = fopen(filename, "wb");
93     if (!file) {
94         fprintf(stderr, "Error '%s'.\n", filename);
95         return;
96     }
97     fwrite(&count, sizeof(int), 1, file);
98
99     for (int i = 0; i < count; i++) {
100         fwrite(&logs[i].id, sizeof(int), 1, file);
101         fwrite(&logs[i].priority, sizeof(int), 1, file);

```

```

102     int message_len = strlen(logs[i].message) + 1;
103     fwrite(&message_len, sizeof(int), 1, file);
104     fwrite(logs[i].message, sizeof(char), message_len, file);
    }
    fclose(file);
}

```

Листинг 5: Исходные коды программы sort (файл: sort.c)

```

1 #include <stdlib.h>
2 #include "heads.h"
3
4 // меняет местами два элемента массива
5 void swap(Log *a, Log *b) {
6     Log temp = *a;
7     *a = *b;
8     *b = temp;
9 }
10
11 // шейкерная сортировка
12 void shaker_sort(Log *logs, int size, int sort_choice, int dir) {
13     int left = 0;
14     int right = size - 1;
15     int swapped = 1;
16
17     while (left < right && swapped) {
18         swapped = 0;
19         // проход слева направо
20         for (int i = left; i < right; i++) {
21             int a, b;
22             if (sort_choice == 0) {
23                 a = logs[i].id;
24                 b = logs[i + 1].id;
25             } else {
26                 a = logs[i].priority;
27                 b = logs[i + 1].priority;
28             }
29
30             if ((dir == 1 && a > b) || (dir == 0 && a < b)) {
31                 swap(&logs[i], &logs[i + 1]);
32                 swapped = 1;
33             }
34         }
35         right--;
36
37         // проход справа налево
38         for (int i = right; i > left; i--) {
39             int a, b;
40             if (sort_choice == 0) {
41                 a = logs[i].id;
42                 b = logs[i - 1].id;
43             } else {
44                 a = logs[i].priority;
45                 b = logs[i - 1].priority;
46             }
47
48             if ((dir == 1 && a < b) || (dir == 0 && a > b)) {
49                 swap(&logs[i], &logs[i - 1]);

```

```

50         swapped = 1;
51     }
52 }
53 left++;
54 }
55 }
56
57 // двухсторонняя сортировка выбором
58 void double_selection_sort(Log *logs, int size, int sort_choice, int dir) {
59     for (int i = 0; i < size / 2; i++) {
60         int min_idx = i;
61         int max_idx = i;
62
63         // МИНИМАЛЬНЫЙ И МАКСИМАЛЬНЫЙ ЭЛЕМЕНТЫ В ОСТАВШЕЙСЯ ЧАСТИ
64         for (int j = i + 1; j < size - i; j++) {
65             int current;
66             int minv;
67             int maxv;
68
69             if (sort_choice == 0) { // по id
70                 current = logs[j].id;
71                 minv = logs[min_idx].id;
72                 maxv = logs[max_idx].id;
73             } else { // по priority
74                 current = logs[j].priority;
75                 minv = logs[min_idx].priority;
76                 maxv = logs[max_idx].priority;
77             }
78             // НОВЫЙ МИНИМУМ
79             if (dir == 1) { // по возрастанию
80                 if (current < minv) {
81                     min_idx = j;
82                 }
83             } else { // по убыванию
84                 if (current > minv) {
85                     min_idx = j;
86                 }
87             }
88
89             // НОВЫЙ МАКСИМУМ
90             if (dir == 1) { // по возрастанию
91                 if (current > maxv) {
92                     max_idx = j;
93                 }
94             } else { // по убыванию
95                 if (current < maxv) {
96                     max_idx = j;
97                 }
98             }
99         }
100         // меняем минимальный элемент с первым неотсортированным
101         swap(&logs[i], &logs[min_idx]);
102         // если максимальный элемент был на месте i, то теперь он находится на
103 min_idx
104         if (max_idx == i) {
105             max_idx = min_idx;
106         }
107         // меняем максимальный элемент с последним неотсортированным
108         swap(&logs[size - i - 1], &logs[max_idx]);

```

```

109     }
110 }
111
112 // сравнитель для qsort (он не умеет сравнивать, поэтому нужны функции-
113 помощники)
114 int srav_qsort_id_voz(const void *a, const void *b) { // id по воз
115     const Log *logA = (const Log *)a;
116     const Log *logB = (const Log *)b;
117     return logA->id - logB->id;
118 }
119
120 int srav_qsort_id_ub(const void *a, const void *b) { // id по уб
121     const Log *logA = (const Log *)a;
122     const Log *logB = (const Log *)b;
123     return logB->id - logA->id;
124 }
125
126 int srav_qsort_priority_voz(const void *a, const void *b) { // priority по воз
127     const Log *logA = (const Log *)a;
128     const Log *logB = (const Log *)b;
129     return logA->priority - logB->priority;
130 }
131
132 int srav_qsort_priority_ub(const void *a, const void *b) { // priority по уб
133     const Log *logA = (const Log *)a;
134     const Log *logB = (const Log *)b;
135     return logB->priority - logA->priority;
136 }
137
138 void qsort_sort(Log *logs, int size, int sort_field, int dir) {
139     if (sort_field == 0) { // сортировка по id
140         if (dir == 1) {
141             qsort(logs, size, sizeof(Log), srav_qsort_id_voz);
142         } else {
143             qsort(logs, size, sizeof(Log), srav_qsort_id_ub);
144         }
145     } else { // сортировка по priority
146         if (dir == 1) {
147             qsort(logs, size, sizeof(Log), srav_qsort_priority_voz);
148         } else {
149             qsort(logs, size, sizeof(Log), srav_qsort_priority_ub);
150         }
151     }
152 }

```

Листинг 6: Исходные коды программы heads (файл: heads.c)

```

1 #ifndef HEADS_H
2 #define HEADS_H
3 #include <stdio.h>
4
5 #define MAXREC 100000
6 #define MAXCHAR 256
7
8 typedef struct {
9     int id;
10    int priority;
11    char message[MAXCHAR];

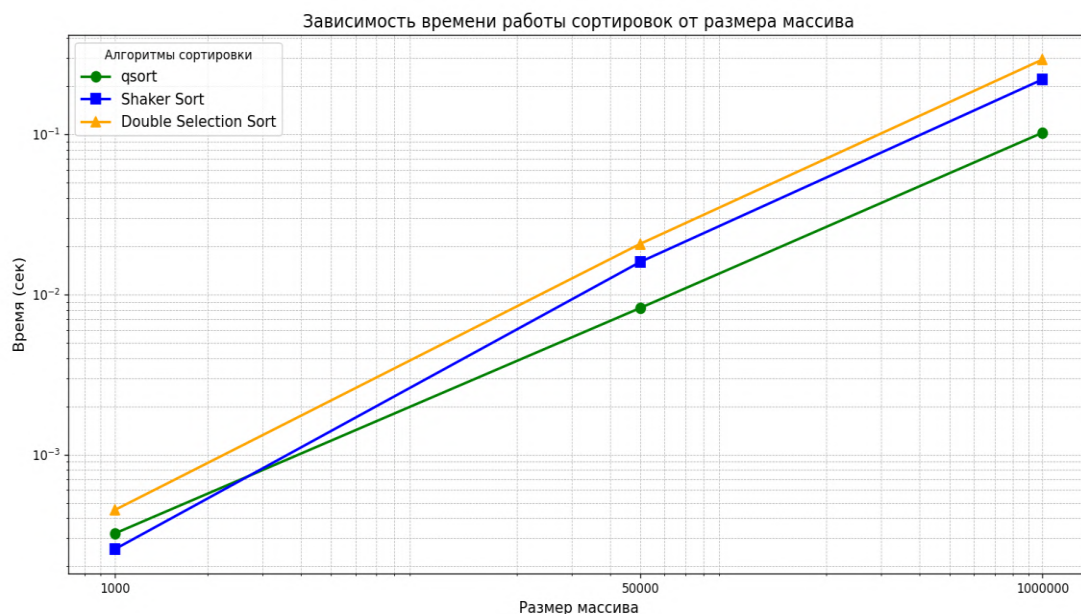
```

```

12 } Log;
13
14 const char* priority_to_string(int priority);
15 int from_stdin(Log *logs, int max_size);
16 int from_text(const char *filename, Log *logs, int max_size);
17 int from_binary(const char *filename, Log *logs, int max_size);
18 void print_logs(const Log *logs, int count);
19 void to_text(const char *filename, const Log *logs, int count);
20 void to_binary(const char *filename, const Log *logs, int count);
21 void shaker_sort(Log *logs, int size, int sort_choice, int dir);
22 void double_selection_sort(Log *logs, int size, int sort_choice, int dir);
23 void qsort_sort(Log *logs, int size, int sort_choice, int dir);
24 void sort_time(void (*sort_func)(Log *, int, int, int), Log *logs, int size, int
25 sort_choice, int dir, const char *sort_name);
26 #endif // HEADS_H

```

5. Графики зависимости времени выполнения сортировок от количества сортируемых элементов



6. Скриншоты

```
[seitnazarov.dn@unix c]$ ls
heads.h main.c rw.c sort.c timer.c
[seitnazarov.dn@unix c]$ gcc main.c rw.c sort.c timer.c -o lab5
[seitnazarov.dn@unix c]$ valgrind ./lab5
==26756== Memcheck, a memory error detector
==26756== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==26756== Using Valgrind-3.24.0 and LibVEX; rerun with -h for copyright info
==26756== Command: ./lab5
==26756==
Enter count of logs: 3
Enter id, priority (0-4) и message: 4
3
oshibka
Enter id, priority (0-4) и message: 6
0
ok
Enter id, priority (0-4) и message: 2
2
jojo

qsort ended for 0.01329600 sec
shaker sort ended for 0.00292300 sec
double selection sort ended for 0.00488700 sec

Logs:
2 [warn] jojo
4 [error] oshibka
6 [debug] ok
==26756==
==26756== HEAP SUMMARY:
==26756==      in use at exit: 0 bytes in 0 blocks
==26756==    total heap usage: 3 allocs, 3 frees, 26,402,048 bytes allocated
==26756==
==26756== All heap blocks were freed -- no leaks are possible
==26756==
==26756== For lists of detected and suppressed errors, rerun with: -s
==26756== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
[seitnazarov.dn@unix c]$
```

Рис.4 : Сборка и запуск программы lab5

7. Выводы

В ходе выполнения данной лабораторной работы были рассмотрены принципы работы с массивами структур и два метода сортировки массивов: Шейкерная сортировка, Двухсторонняя сортировка выбором. Проведены исследования и построен график зависимости времени выполнения программы от кол-ва элементов и различных видов сортировок.